Gusto Meets (Working Title) - Comprehensive Business & Technical Blueprint

1. Business Requirement Document (BRD)

Project Name: Gusto Meets (Working Title) **Core Vision:** To create an asset-light, two-sided marketplace (Progressive Web App) that monetizes underutilized residential terraces by renting them hourly for private, structured socializing, bridging the gap between restrictive households and expensive commercial cafes.

Key Business Rules & Mechanics:

- **Intermediary Status:** The platform operates strictly as a digital aggregator (Safe Harbor under India IT Act Section 79). It does not own the real estate and shifts physical liability to the host and guest via digital affidavits and waivers.
- **Airbnb-Style Onboarding:** Hosts curate their space listings through rich media and strict "Permission Matrices" (e.g., alcohol allowed, couples allowed, no loud music).
- **Zoomcar-Style Outlier Management:** The system handles rule-breaking (overstays, damages) via mandatory wallet-based security deposits and automated penalty deductions, completely removing the host from awkward financial confrontations.

---

2. System Actors

1. **Guest (Renter):** Discovers spaces, undergoes KYC verification, loads the platform wallet, and books terraces by the hour.
2. **Host (Terrace Owner):** Lists the property, defines the permission matrix, uploads safety proofs, physically/digitally verifies the guest's Aadhaar at check-in, and receives payouts.
3. **Platform Admin:** Manages dispute resolution (arbitration for damages/overstays), audits host safety videos, and manages platform configurations.
4. **System (Automated):** Handles the No-OTP Aadhaar API calls, triggers "extend or evacuate" notifications, deducts overstay penalties, and dynamically updates inventory.

---

3. Detailed Functional Requirements (FR)

FR1: Airbnb-Style Host Onboarding & Listing

- **FR1.1 Property Tagging:** Hosts must tag the property access type: Private External Staircase or Shared Walk-through.
- **FR1.2 Permission Matrix:** Hosts toggle allowed activities (Dine-out, Board Games, Alcohol, Smoking, Birthday Parties).
- **FR1.3 Safety Verification (Rich Media):** Hosts must upload an uncut 60-second video walkthrough demonstrating a minimum 4-foot parapet wall and lighting.
- **FR1.4 Digital Affidavit:** Hosts must digitally sign a liability waiver stating they are legally authorized to rent the space and that safety features are accurately represented.

FR2: Guest Onboarding & No-OTP Aadhaar KYC

- **FR2.1 Aadhaar Input & Verhoeff Check:** The guest enters their Aadhaar number. The frontend immediately runs a local Verhoeff algorithm check to ensure the numeric string is mathematically valid before making an API call.
- **FR2.2 No-OTP API Gateway Validation:** The system sends a secure POST request to a third-party KYC gateway. The gateway checks the Central Identity Data Repository (CIDR) to confirm the ID exists without triggering a mobile OTP.
- **FR2.3 Fuzzy Name Matching:** The system matches the name inputted by the guest against the demographic data returned by the API to ensure high-confidence identity verification.
- **FR2.4 Host Verification at Check-in:** The guest's verified status and demographic details are pushed to the Host's dashboard. The Host conducts a visual confirmation of the guest against the uploaded ID at the terrace entrance.

FR3: Zoomcar-Style Booking & Outlier Handling

- **FR3.1 Purpose-Driven Dynamic Pricing:** Base hourly rates scale based on the selected activity (e.g., a "Party" booking carries a 30% premium over a "Movie Night" booking).
- **FR3.2 Mandatory Security Deposit:** High-impact bookings require a refundable security deposit authorized on the guest's payment method, held in escrow.
- **FR3.3 The "Extend or Evacuate" Trigger:** At T-minus 15 minutes to session end, the system sends an automated SMS/Push. Guests can click to extend (auto-deducting from wallet) or confirm checkout.
- **FR3.4 Overstay Penalties:** If a guest overstays their booked slot without extending, the system automatically deducts a punitive fee (e.g., 2x the hourly rate) from their security deposit/wallet and compensates the host or the delayed subsequent guest.

---

4. Detailed Non-Functional Requirements (NFR)

- **Security & Privacy:**
  - No Aadhaar numbers are stored in plain text. Once the No-OTP verification passes, the ID is tokenized/hashed, and only the "Verified Name" and "Verification Status" are stored.
  - Data encrypted at rest (AES-256) and in transit (TLS 1.3).
- **Performance:** The PWA must achieve a First Contentful Paint (FCP) of < 1.5 seconds on mobile 4G networks to prevent drop-offs during the QR-scan walk-in flow.
- **Scalability:** The booking engine must handle high concurrent read/writes during peak weekend hours (Friday 6 PM – Sunday 11 PM) using database connection pooling.
- **Availability:** 99.9% uptime, particularly for the payment gateway and KYC API integrations, as downtime directly blocks user entry to physical spaces.

---

5. Mobile Application Architecture (PWA)

Because you need to avoid the friction of an App Store download for casual users, a Progressive Web App (PWA) is the optimal architecture.

Frontend Layer (Client-Side):

- **Framework:** Next.js (React) configured as a PWA (manifest.json, service workers for offline caching of UI elements).
- **Styling:** Tailwind CSS for lightweight, responsive, mobile-first design.

- **State Management:** Zustand or React Context for managing the "Wallet Balance" and active booking timers.

Backend Layer (Server-Side):

- **Framework:** Node.js with FastAPI (Python) microservices for the heavy computational tasks (like fuzzy name matching and Verhoeff validation).
- **Database:** PostgreSQL (via Supabase) for relational data (Users, Terraces, Bookings) and real-time WebSockets to update the host's dashboard the second a guest pays.
- **Auth:** Supabase Auth (Phone number + OTP login).

Third-Party Integrations:

- **Payments & Escrow:** Razorpay API (handles UPI intent, wallet top-ups, and pre-authorizations for security deposits).
- **KYC Gateway:** API provider (e.g., Zoop.one or Setu) for the No-OTP Aadhaar demographic validation.
- **Communications:** WhatsApp Business API (for instant booking confirmations, location sharing, and overstay warnings).

---

6. Detailed Mobile Application Design (UX Flow)

Phase 1: Discovery & Booking (Guest Flow)

1. **Landing & Search:** User opens PWA. Enters location and group size.
2. **Filter & Match:** User filters by "Permission" (e.g., Show me places that allow smoking and outside food).
3. **Property Details:** Views photos, host rules, and access type. Selects a time slot.
4. **KYC Checkpoint:** If a first-time user, they hit the No-OTP Aadhaar flow. Enters Name and ID. Validates in < 3 seconds.
5. **Checkout & Deposit:** User pays the hourly fee + security deposit via UPI.
6. **Confirmation & Access:** User receives a WhatsApp message with Google Maps pin, exact access instructions (e.g., "Take the iron staircase on the left"), and the Host's contact number.

Phase 2: Active Session (In-Life Flow)

1. **Check-in:** Guest arrives. Host verifies ID on their app dashboard. Host hits "Check-in Guest".
2. **The Live Timer:** Guest's app shows a countdown timer of their remaining time.
3. **T-Minus 15 Alert:** Screen flashes. Option: "Add 1 Hour (₹200)" or "Checkout".

Phase 3: Dispute & Exception Design (Zoomcar Flow)

1. **Host Post-Checkout:** Host inspects the terrace. If there are cigarette burns or broken glass, Host opens the app, clicks "Report Damage", and uploads photos.
2. **System Action:** The system freezes the Guest's security deposit.
3. **Admin Arbitration:** Admin reviews the photos against the pre-booking baseline. Admin approves the claim.
4. **Resolution:** The penalty amount is captured from the Razorpay authorization and routed to the Host. The remainder is refunded to the Guest.

---

7. PostgreSQL Database Schema

This schema translates your business rules into strict database constraints specifically designed for the Next.js/Supabase architecture.

```
-- ================================================================================
-- 1. ENUMS (Custom Data Types for Strict Validation)
-- ================================================================================

CREATE TYPE user_role AS ENUM ('GUEST', 'HOST', 'ADMIN');
CREATE TYPE access_type AS ENUM ('PRIVATE_STAIRCASE', 'SHARED_WALKTHROUGH');
CREATE TYPE booking_purpose AS ENUM ('CHILLOUT', 'DINE_OUT', 'MOVIE_NIGHT',
'PARTY', 'DANCE_PRACTICE');
CREATE TYPE booking_status AS ENUM ('PENDING_PAYMENT', 'CONFIRMED', 'ACTIVE',
'COMPLETED', 'OVERSTAYED', 'DISPUTED', 'CANCELLED');

-- ================================================================================
-- 2. USERS TABLE
-- ================================================================================

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone_number VARCHAR(15) UNIQUE NOT NULL,
    full_name VARCHAR(100),
    role user_role DEFAULT 'GUEST',

    -- KYC & Trust (Zero-Knowledge Architecture)
    kyc_verified BOOLEAN DEFAULT FALSE,
    kyc_reference_token VARCHAR(255), -- Stores the API gateway transaction ID,
NOT the Aadhaar number
    wallet_balance DECIMAL(10, 2) DEFAULT 0.00,

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ================================================================================
-- 3. TERRACES TABLE (Airbnb-Style Listings & Safety)
-- ================================================================================

CREATE TABLE terraces (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    host_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    title VARCHAR(150) NOT NULL,
    description TEXT,
    address_line TEXT NOT NULL,
    geo_location POINT, -- For spatial queries (distance from user)

    -- Architecture & Access
    access_category access_type NOT NULL,
    max_capacity INT NOT NULL CHECK (max_capacity > 0 AND max_capacity <= 50),
```

```
    -- Economics
    base_hourly_rate DECIMAL(10, 2) NOT NULL CHECK (base_hourly_rate > 0),

    -- Safety & Due Diligence (Legal Shield)
    parapet_height_ft DECIMAL(4, 2) NOT NULL CHECK (parapet_height_ft >= 4.0),
    safety_video_url VARCHAR(255) NOT NULL, -- Uncut 60s walkthrough
    is_affidavit_signed BOOLEAN DEFAULT FALSE NOT NULL,
    is_active BOOLEAN DEFAULT FALSE, -- Requires Admin approval first

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ================================================================================
-- 4. PERMISSION MATRICES (The Smart Matching Engine)
-- ================================================================================

CREATE TABLE terrace_permissions (
    terrace_id UUID PRIMARY KEY REFERENCES terraces(id) ON DELETE CASCADE,

    -- Boolean Toggles (What is allowed?)
    allow_alcohol BOOLEAN DEFAULT FALSE,
    allow_smoking BOOLEAN DEFAULT FALSE,
    allow_loud_music BOOLEAN DEFAULT FALSE,
    allow_outside_food BOOLEAN DEFAULT TRUE, -- Core BYOF feature

    -- Allowed Purposes (Array of ENUMs for easy filtering)
    allowed_purposes booking_purpose[] NOT NULL,

    -- Dynamic Pricing Multipliers
    party_rate_multiplier DECIMAL(3, 2) DEFAULT 1.00 CHECK (party_rate_multiplier
>= 1.0),
    alcohol_deposit_required DECIMAL(10, 2) DEFAULT 0.00,

    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ================================================================================
-- 5. BOOKINGS TABLE (Zoomcar Outlier Handling & State Machine)
-- ================================================================================

CREATE TABLE bookings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    guest_id UUID NOT NULL REFERENCES users(id),
    terrace_id UUID NOT NULL REFERENCES terraces(id),

    -- Time Logistics
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ NOT NULL,
    actual_checkout_time TIMESTAMPTZ, -- Populated when guest leaves

    -- Booking Metadata
    purpose booking_purpose NOT NULL,
```

```
    guest_count INT NOT NULL CHECK (guest_count > 0),
    status booking_status DEFAULT 'PENDING_PAYMENT',

    -- Financials (Locked in at time of booking to prevent historical changes)
    hourly_rate_applied DECIMAL(10, 2) NOT NULL,
    total_time_cost DECIMAL(10, 2) NOT NULL,
    security_deposit_held DECIMAL(10, 2) DEFAULT 0.00,

    -- Zoomcar Penalty Handling
    overstay_penalty_applied DECIMAL(10, 2) DEFAULT 0.00,
    damage_penalty_applied DECIMAL(10, 2) DEFAULT 0.00,

    -- Transaction Links
    razorpay_order_id VARCHAR(100),
    digital_waiver_signed BOOLEAN DEFAULT FALSE NOT NULL,

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),

    -- Integrity Constraint: End time must be strictly after Start time
    CONSTRAINT valid_time_range CHECK (end_time > start_time)
);

-- ============================================================================
-- 6. CRITICAL INDEXES (For Read Performance)
-- ============================================================================

-- Optimize location-based and capacity searches
CREATE INDEX idx_terraces_active_capacity ON terraces(is_active, max_capacity);

-- Optimize checking for overlapping bookings (Double-booking prevention)
CREATE INDEX idx_bookings_time_range ON bookings(terrace_id, start_time, end_time)
WHERE status IN ('CONFIRMED', 'ACTIVE', 'OVERSTAYED');

-- Optimize finding a guest's active or upcoming bookings
CREATE INDEX idx_bookings_guest_status ON bookings(guest_id, status);
```

---

8. Resolution of Core Challenges

- **The RWA / Neighbor Filtering Problem:** * *Query:* `SELECT terrace_id FROM terrace_permissions WHERE 'PARTY' = ANY(allowed_purposes) AND allow_alcohol = TRUE;`
    - *Result:* The quiet, retired host never even sees the request, preventing complaints.
- **The Zoomcar Damage/Overstay Problem:** * When a booking goes to 'OVERSTAYED', a webhook fires, calculates the extra time, writes it to `overstay_penalty_applied`, and deducts it from the `security_deposit_held`. The host never has to physically confront the guest for cash.
- **The Double-Booking Prevention:** * The index `idx_bookings_time_range` enables the backend to instantly check if a requested `start_time` and `end_time` overlap with any existing confirmed booking.